

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Spring Boot Actuator



Salitha Chathuranga · [Follow](#)

6 min read · Aug 21, 2022

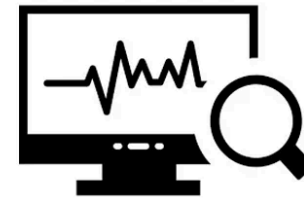


51



Let's monitor Spring Boot applications

SPRING BOOT ACTUATOR DETAILED GUIDE



Hey guys! I'm back with another Spring Boot article 😎 If you missed the previous articles, here we go. 🙋

 Salitha Chathuranga

Spring Boot Series

View list

13 stories

Spring
BOOT



20

INTEGRATE TE
CONTAINERS \
SPRING BOOT

LET'S TEST OUR CO

JAVA / MONGODB
INTEGRATION/FUNCTION
TEST DRIVEN DEVELOPMI
TEST DATA SIMULATION

IMPLEMENT C
MONGO CLIEN

REPOSITORY LAYE
SPRING BOOT
MONGODB JAVA
TEST INTEGRATIO
TEST CONTAINERS

TIONAL
WITH
DOT

BERNATE

• ONE
• ONE
• MANY

Spring Boot Actuator is sub module of Spring framework which **enables the access to operational insights, metric of a running application**. There is a bunch of endpoints already listed inside the module. We can even monitor CPU usage. JVM usage taking by the application. Not only that, we can get application information, database information, environment information also. We can setup the project to trace requests received to the back-end! 🤖

Simply, we can create a monitoring application using these endpoints which is used by the internal teams. ❤️

Setup Spring Boot Actuator

I will create a simple REST API project to demonstrate the things. I have connected MySQL database through JPA and hibernate in this project. Since **actuator** is coming with a **starter dependency**, we have to add it to the POM to get its functionalities.

```
<dependency>  
  <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

After adding this dependency and re running the project, you will see some log while application is starting saying that actuator is ready to use.

```
2022-08-11 20:25:26.511 INFO 22607 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2022-08-11 20:25:26.747 INFO 22607 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2022-08-11 20:25:26.760 INFO 22607 --- [main] org.hibernate.dialect.MySQL8Dialect : HHH000400: Using dialect: org.hibernate.dialect.MySQL8Dialect
2022-08-11 20:25:26.892 INFO 22607 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
2022-08-11 20:25:26.899 INFO 22607 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2022-08-11 20:25:26.956 WARN 22607 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2022-08-11 20:25:27.312 INFO 22607 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint(s) beneath base path '/actuator'
2022-08-11 20:25:27.356 INFO 22607 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s) 8080 (http) with context path ''
2022-08-11 20:25:27.374 INFO 22607 --- [main] c.r.actuatorapp.ActuatorAppApplication : Started ActuatorAppApplication in 3.152 seconds (JVM running for 3.463)
2022-08-11 20:25:44.028 INFO 22607 --- [nio-8080-exec-1] o.a.c.c.([Tomcat].[localhost].[/]) : Initializing Spring DispatcherServlet 'dispatcherServlet'
2022-08-11 20:25:44.028 INFO 22607 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2022-08-11 20:25:44.029 INFO 22607 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
2022-08-11 20:28:28.330 INFO 22607 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2022-08-11 20:28:28.331 INFO 22607 --- [ionShutdownHook] .SchemaDropperImpl$DelayedDropActionImpl : HHH000477: Starting delayed evictData of schema as part of SessionFactory shut-down'
```

Spring Boot actuator log

Visit [here](http://localhost:8080/actuator) to check already enabled actuator APIs by default:

<http://localhost:8080/actuator>. The default endpoint configuration is like this. Only few endpoints are exposed.

```
{
  "_links":{
    "self":{
      "href":"http://localhost:8080/actuator",
      "templated":false
    },
    "health":{
```

```

        "href": "http://localhost:8080/actuator/health",
        "templated": false
    },
    "health-path": {
        "href": "http://localhost:8080/actuator/health/{*path}",
        "templated": true
    }
}
}

```

Enable all actuator endpoints

We have to give a **config property** to **enable all the other endpoints** inside **application.properties** file.

```
management.endpoints.web.exposure.include=*
```

Then all the endpoints will be shown when we visit the actuator link.

```

{
  "_links": {
    "self": {

```

```
    "href": "http://localhost:8080/actuator",
    "templated": false
  },
  "beans": {
    "href": "http://localhost:8080/actuator/beans",
    "templated": false
  },
  "caches-cache": {
    "href": "http://localhost:8080/actuator/caches/{cache}",
    "templated": true
  },
  "caches": {
    "href": "http://localhost:8080/actuator/caches",
    "templated": false
  },
  "health": {
    "href": "http://localhost:8080/actuator/health",
    "templated": false
  },
  "health-path": {
    "href": "http://localhost:8080/actuator/health/{*path}",
    "templated": true
  },
  "info": {
    "href": "http://localhost:8080/actuator/info",
    "templated": false
  },
  "conditions": {
    "href": "http://localhost:8080/actuator/conditions",
    "templated": false
  },
  "configprops": {
    "href": "http://localhost:8080/actuator/configprops",
    "templated": false
  },
  "configprops-prefix": {
    "href": "http://localhost:8080/actuator/configprops/{prefix}",
    "templated": true
  }
```

```
,
"env-toMatch": {
  "href": "http://localhost:8080/actuator/env/{toMatch}",
  "templated": true
},
"env": {
  "href": "http://localhost:8080/actuator/env",
  "templated": false
},
"loggers": {
  "href": "http://localhost:8080/actuator/loggers",
  "templated": false
},
"loggers-name": {
  "href": "http://localhost:8080/actuator/loggers/{name}",
  "templated": true
},
"heapdump": {
  "href": "http://localhost:8080/actuator/heapdump",
  "templated": false
},
"threaddump": {
  "href": "http://localhost:8080/actuator/threaddump",
  "templated": false
},
"metrics-requiredMetricName": {
  "href":
"http://localhost:8080/actuator/metrics/{requiredMetricName}",
  "templated": true
},
"metrics": {
  "href": "http://localhost:8080/actuator/metrics",
  "templated": false
},
"scheduledtasks": {
  "href": "http://localhost:8080/actuator/scheduledtasks",
  "templated": false
},
```

```
"mappings": {  
  "href": "http://localhost:8080/actuator/mappings",  
  "templated": false  
}  
}
```

Enable detailed Health endpoint

By default health endpoint will only contain the status only. if you go here: <http://localhost:8080/actuator/health> you would get response as this.

```
{  
  "status": "UP"  
}
```

It is not containing enough detail right? Now add the below **config** in your **application.properties** file to enable detailed API response for health.

```
management.endpoint.health.show-details=always
```


Now you will get a detailed view for health endpoint... 😎

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "MySQL",
        "validationQuery": "isValid()"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 101057732608,
        "free": 43956203520,
        "threshold": 10485760,
        "exists": true
      }
    },
    "ping": {
      "status": "UP"
    }
  }
}
```

Enable Application Information endpoint

By default, `/info` endpoint is disabled in actuator. We need to add the below config to `application.properties` file to see application related information exposed by us.

```
management.info.env.enabled=true
info.app.name=Spring Boot Actuator Dashboard
info.app.description=Spring Boot backend to demonstrate actuator APIs
info.app.version=1.0
```

So, after adding this visit here: <http://localhost:8080/actuator/info> to get application metadata.

```
{
  "app": {
    "name": "Spring Boot Actuator Dashboard",
    "description": "Spring Boot backend to demonstrate actuator APIs",
    "version": "1.0"
  }
}
```

Enable HTTP Trace endpoint

We can trace the application requests also using Actuator. But in recent versions of Spring Boot, it is disabled by default. So, we can enable that by adding the below config to **application.properties**.

```
management.trace.http.enabled=true
```

This is not enough guys! 😊 We have add a bean configuration also in addition. You must add the below Configuration to the project. I have named the class as **HttpTraceActuatorConfiguration**.

```
package com.example.demoapp.configs;

import
org.springframework.boot.actuate.trace.http.HttpTraceRepository;
import
org.springframework.boot.actuate.trace.http.InMemoryHttpTraceRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
public class HttpTraceActuatorConfiguration {

    @Bean
    public HttpTraceRepository httpTraceRepository() {
        return new InMemoryHttpTraceRepository();
    }

}
```

Now, go and visit here: <http://localhost:8080/actuator/httptrace>. What do you see there? You should be having some kind of a response given below.

```
▼ {
  ▼ "traces": [
    ▼ {
      "timestamp": "2022-08-11T16:19:16.006Z",
      "principal": null,
      "session": null,
      ▶ "request": { ... }, // 4 items
      ▶ "response": { ... }, // 2 items
      "timeTaken": 127
    },
    ▼ {
      "timestamp": "2022-08-11T16:19:02.670Z",
      "principal": null,
      "session": null,
      ▶ "request": { ... }, // 4 items
      ▼ "response": {
        "status": 200,
        ▼ "headers": {
          ▼ "Keep-Alive": [
            "timeout=60"
          ],
          ▼ "Transfer-Encoding": [
            "chunked"
          ],
          ▼ "Connection": [
            "keep-alive"
          ],
          ▼ "Date": [
            "Thu, 11 Aug 2022 16:19:02 GMT"
          ],
          ▼ "Content-Type": [
            "application/vnd.spring-boot.actuator.v3+json"
          ]
        }
      },
      "timeTaken": 254
    }
  ]
}
```

Now you can see all your requests in detail. Actually with these APIs, you can create an **internally used monitoring dashboard front-end app** for your back-end services 🥰🤗

Change actuator Base path

Add the below **config** in your **application.properties** file to change actuator endpoint path. This will replace <http://localhost:8080/actuator> with <http://localhost:8080/manage>.

```
management.endpoints.web.base-path=/manage
```

Check system related Metrics

We can get a lot of details about the running application using **/metrics** endpoints. If you go here: <http://localhost:8080/actuator/metrics> you will get all the metrics available.

```
{
  "names": [
    "application.ready.time",
    "application.started.time",
    "disk.free",
    "disk.total",
    "executor.active",
    "executor.completed",
    "executor.pool.core",
    "executor.pool.max",
    "executor.pool.size",
    "executor.queue.remaining",
    "executor.queued",
    "hikaricp.connections",
    "hikaricp.connections.acquire",
    "hikaricp.connections.active",
    "hikaricp.connections.creation",
    "hikaricp.connections.idle"
```



🔍 Search

✍ Write



```
"hikaricp.connections.pending",
    "hikaricp.connections.timeout",
    "hikaricp.connections.usage",
    "http.server.requests",
    "jdbc.connections.max",
    "jdbc.connections.min",
    "jvm.buffer.count",
    "jvm.buffer.memory.used",
    "jvm.buffer.total.capacity",
```

```
"jvm.classes.loaded",  
"jvm.classes.unloaded",  
"jvm.gc.live.data.size",  
"jvm.gc.max.data.size",  
"jvm.gc.memory.allocated",  
"jvm.gc.memory.promoted",  
"jvm.gc.overhead",  
"jvm.gc.pause",  
"jvm.memory.committed",  
"jvm.memory.max",  
"jvm.memory.usage.after.gc",  
"jvm.memory.used",  
"jvm.threads.daemon",  
"jvm.threads.live",  
"jvm.threads.peak",  
"jvm.threads.states",  
"logback.events",  
"process.cpu.usage",  
"process.files.max",  
"process.files.open",  
"process.start.time",  
"process.uptime",  
"spring.data.repository.invocations",  
"system.cpu.count",  
"system.cpu.usage",  
"system.load.average.1m",  
"tomcat.sessions.active.current",  
"tomcat.sessions.active.max",  
"tomcat.sessions.alive.max",  
"tomcat.sessions.created",  
"tomcat.sessions.expired",  
"tomcat.sessions.rejected"  
]  
}
```


You can see any of these APIs using **/metrics** prefix with relevant metric name. The most important metric APIs to track:

<http://localhost:8080/actuator/metrics/system.cpu.usage>
<http://localhost:8080/actuator/metrics/jvm.memory.max>
<http://localhost:8080/actuator/metrics/jvm.memory.used>
<http://localhost:8080/actuator/metrics/process.uptime>
<http://localhost:8080/actuator/metrics/application.started.time>
<http://localhost:8080/actuator/metrics/disk.free>
<http://localhost:8080/actuator/metrics/disk.total>
<http://localhost:8080/actuator/metrics/hikaricp.connections>

```
▼ {
  "name": "system.cpu.usage",
  "description": "The \"recent cpu usage\" of the system the application is running in",
  "baseUnit": null,
  ▼ "measurements": [
    ▼ {
      "statistic": "VALUE",
      "value": 0.11126005361930295
    }
  ],
  "availableTags": []
}
```

CPU usage response

```

{
  "name": "jvm.memory.used",
  "description": "The amount of used memory",
  "baseUnit": "bytes",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 122733464
    }
  ],
  "availableTags": [
    {
      "tag": "area",
      "values": [
        "heap",
        "nonheap"
      ]
    },
    {
      "tag": "id",
      "values": [
        "Compressed Class Space",
        "PS Survivor Space",
        "PS Old Gen",
        "Metaspace",
        "PS Eden Space",
        "Code Cache"
      ]
    }
  ]
}

```

JVM Memory usage response

Create a custom actuator endpoint 🌟

We can create custom logic and return as a new actuator endpoint settled in actuator. There we need to setup a “@Component” with the relevant method logic. I named endpoint as “custom” using `Endpoint` annotation. It will be resolved later as <http://localhost:8080/actuator/custom>. The class name will be `CustomEndpoint` here.

```
@Endpoint(id = "custom")
@Component
public class CustomEndpoint {
    @ReadOperation
    public Map<String, String> customEndpoint() {
        Map<String, String> map = new HashMap<>();
        map.put("k1", "v1");
        map.put("k2", "v2");
        return map;
    }
}
```

I have just returned a dummy key value map in the code as the method logic. Then the endpoint will be listed inside actuator links as given below when you re run the application.

```
▼ {
  ▼ "_links": {
    ▼ "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    ▼ "custom": {
      "href": "http://localhost:8080/actuator/custom",
      "templated": false
    },
    ▼ "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    ▼ "caches-cache": {
      "href": "http://localhost:8080/actuator/caches/{cache}",
      "templated": true
    },
  },
}
```

Listed endpoint within actuator links

Visit the link and see the result....

```
{
  "k1": "v1",
  "k2": "v2"
}
```

Update already available endpoint logic programmatically 🌟

Actuator APIs can be accessed via the code also. It provides us the ability to do it in programmatic way also. Here I will take health endpoint and update its logic. Currently its response has this keys: **db**, **diskSpace** and **ping**. I'm going to add another key here..

There we need to access health endpoint interface and override the logic! I have created a new class called **LoggerService**. When this component is added to the health, it will take class name to setup the key to the response.

```
@Component
public class LoggerService implements HealthIndicator {
    @Override
    public Health health() {
        if (isLoggingEnabled()) return
Health.up().withDetail("logging", "Loggers are enabled").build();
        return Health.down().withDetail("logging", "Loggers are
disabled").build();
    }

    private boolean isLoggingEnabled() {
        return true; // dummy - this is the place we need a proper
implementation.
    }
}
```

Look at the response below. It has “loggerService” key in the response.

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "MySQL",
        "validationQuery": "isValid()"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 101057732608,
        "free": 51914203136,
        "threshold": 10485760,
        "exists": true
      }
    },
    "loggerService": {
      "status": "UP",
      "details": {
        "logging": "Loggers are enabled"
      }
    },
    "ping": {
      "status": "UP"
    }
  }
}
```

Modified health endpoint view

I think I have explained everything I know about **Actuator**. you can setup these endpoints in your **Spring Boot micro services**. Then you can see the application metrics easily. If you want, you can optimize the code then looking at the metrics like JVM usage / CPU usage.

Bye bye readers! Stay safe! ❤️

Spring Boot

Actuator

Java

Spring Web

Spring Framework

